ЛАБОРАТОРНА РОБОТА № 7

Fetch API та MockAPI

Виконав: Мельник Назар
Група: КН-310
Індивідуальний номер: 14

Львів — 2026

1. Тема та мета

Тема: завантаження даних із публічного API та відображення на сторінці.

Мета: навчитися виконувати асинхронні HTTP-запити через Fetch API, опрацьовувати JSON, реалізувати CRUD-операції для ресурсу MockAPI, оновлювати DOM та коректно повідомляти про помилки.

2. Завдання

Для USER_ID = 14 отримати записи todoItem, відобразити їх у DOM, реалізувати POST, PUT і DELETE. Перевірити HTTP-статуси, обробку помилок та формат дати.

3. Короткі теоретичні відомості

fetch() повертає Promise з об'єктом Response. Мережева помилка відхиляє Promise, але HTTP-помилки треба окремо визначати через response.ok або response.status. response.json() асинхронно перетворює JSON-відповідь на JavaScript-значення. У роботі застосовано async/await, try/catch, Content-Type: application/json і JSON.stringify().

4. Реалізація

API: https://69b11abdadac80b427c3fff2.mockapi.io/api/v1/todoItem

GET отримує список і фільтрує його за Number(item.userId) === 14. Форма передає JSON із userId, title, description, tag, isDone та deadline. Після POST/PUT список завантажується повторно. Картка вилучається з DOM тільки після успішного DELETE. Перед зміною перевіряється належність запису користувачу №14.

formatDeadline() підтримує ISO-дати та Unix timestamp у секундах або мілісекундах. Календарно некоректні ISO-дати, порожні та інші некоректні значення обробляються без помилки; додатково перевірено 14 тестових випадків.

5. Фактична перевірка CRUD

Playwright запущено на локальному HTTP-сервері у headless Chromium із реальним MockAPI. Створено тільки тимчасовий запис LR07_AUDIT_1793534238014_POST, ID 9; відредаговано лише його, після тесту видалено.

Вимога	Реалізація	Перевірка	Доказ
READ	GET + USER_ID	200, PASS	Рис. 1
CREATE	POST, JSON	201, ID 9, PASS	Рис. 2
UPDATE	PUT /todoItem/9	200, DOM PASS	Рис. 3
DELETE	DELETE /todoItem/9	200, absent PASS	Рис. 4
Дата	formatDeadline()	2026-11-05; 14/14	Рис. 2–3
Помилки	ok + try/catch	0 browser errors	console log

Фактичні статуси: GET — 200, POST — 201, PUT — 200, DELETE — 200. Повторні GET — 200.

6. Аналіз NetworkError

Попередній звіт містив одиничне повідомлення Firefox: TypeError: NetworkError when attempting to fetch resource. Під час фінального повтору API був доступний, усі CRUD-запити завершилися успішно. Access-Control-Allow-Origin: * і дозволені методи підтверджують відсутність CORS-блокування в цьому запуску. Консоль браузера порожня, requestfailed немає.

Під час повторної перевірки не відтворено.

За одним попереднім повідомленням конкретну первинну причину встановити неможливо.

7. Докази

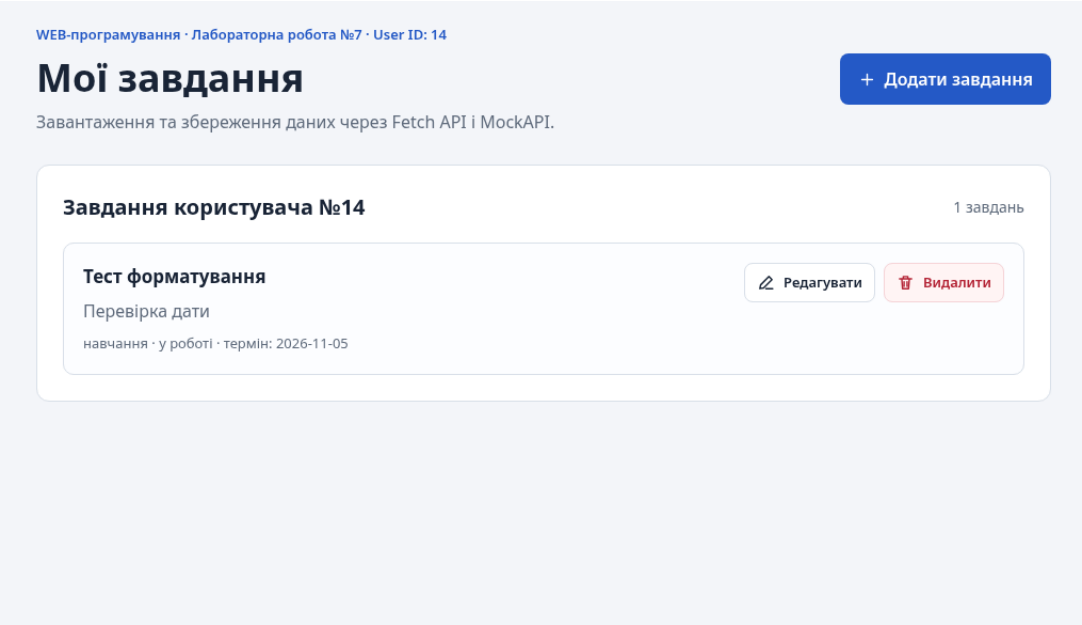


Рисунок 1 — Початкове завантаження та дані користувача №14 після GET (200).

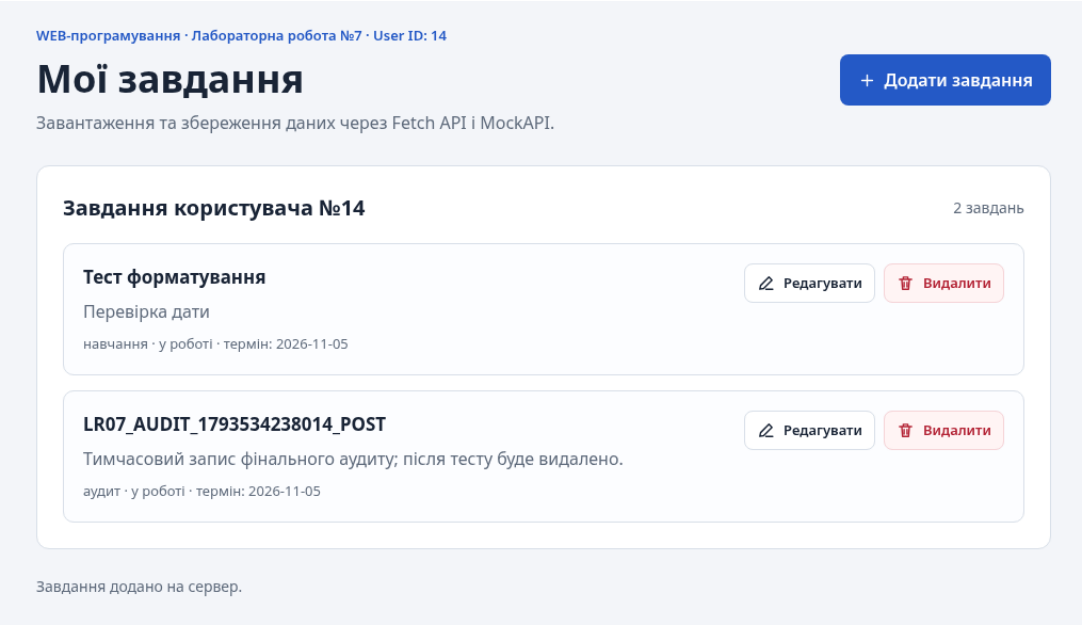


Рисунок 2 — Тимчасовий запис після POST (201); дата 2026-11-05.

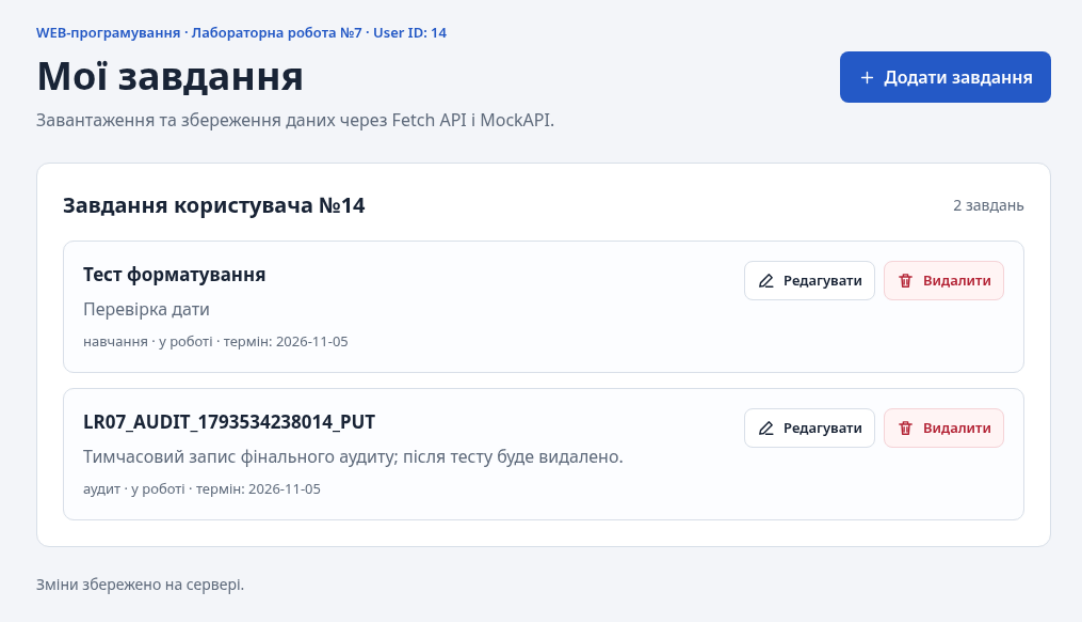


Рисунок 3 — Запис після PUT (200); дата збережена.

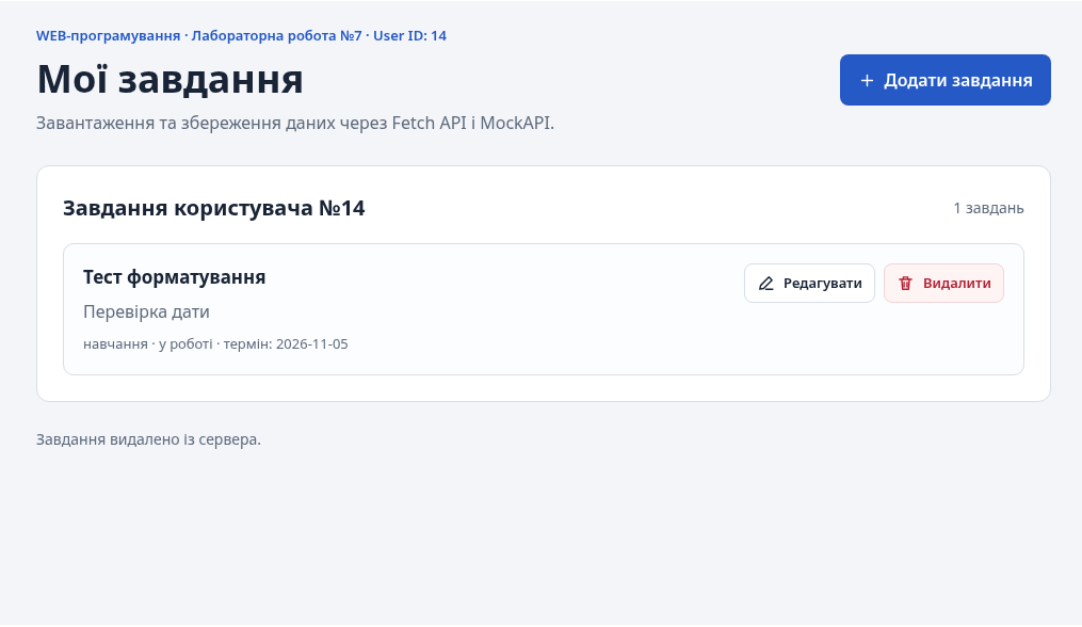


Рисунок 4 — Після DELETE (200); повторний GET підтвердив відсутність.

8. Висновок

CRUD через Fetch API для користувача №14 підтверджено реальним сервером. За початковим журналом Codex пройдено 12/12 браузерних перевірок і 8/8 тестів дати. Після додаткового виправлення календарної валідації отримано 14/14 локальних тестів дати й успішну перевірку синтаксису JavaScript. Тимчасовий ID 9 видалено; чужі й раніше створені записи не змінювалися. Під час незалежної перевірки усунено граничний випадок: календарно некоректні ISO-дати тепер відхиляються. HTTP-журнали та скриншоти належать первинному запуску Codex; після цього виправлення живий CRUD повторно не запускався.